HEX [PI] 7.0

The mathematics, the code that computes it, and the learning loop

Anna Maria Sarkisyan · Evrica Technologies Inc.
5 October 2026 · engine version 7.0 on HEX6-BLE · open source

How to read this. Every step appears three times. The shaded box says what the step does and why, in ordinary language, with a worked number. The text under it states the same thing exactly. Then the code that computes it is printed as it stands in the repository, with its file and line numbers, so that the equation and the implementation can be checked against each other. The engine's Python reference and this TypeScript agree to 10^{-13} on identical inputs (`tests/parity.mjs`).

The whole engine is one equation and one loop:

$$\boxed{\operatorname{logit} p_t = \operatorname{logit} q_{0,t} + \beta_t^\top Z_t} \qquad \boxed{\text{reveal} \to \text{score} \to \text{learn} \to \text{predict} \to \text{commit}}$$

Symbol	In plain words	Code
q_0	the null: what the series' own record already implies, its usual odds	`observer.ts`
Z	where the present value sits among its surroundings	`hex7.ts: ranks`
β	what HEX has learned on this series	`hex7.ts: Residual`
p	HEX's call	`hex7.ts: walkHex7`

1 The data and the two questions

In plain words

The engine never looks at raw values. It looks at how much each value changed from the one before, and how big that change was compared with what is usual for the series. It asks two yes-or-no questions about the next step: *will the value go up?* and *will the change be bigger than usual?*

An adapter turns a series $x_0, x_1, \ldots$ into changes r_t: $\ln(x_t/x_{t-1})$ when every value is positive, $x_t - x_{t-1}$ otherwise (`probabilityRead.ts: seriesToReturns`). Everything after the adapter is the same for every kind of series. With $a_t = |r_t|$ and $m_t = \text{median}(a_1, \ldots, a_t)$ for $t \geq 20$:

$$\text{MOVE}: \; y_{t+1} = \mathbb{1}[r_{t+1} > 0], \qquad \text{RANGE}: \; y_{t+1} = \mathbb{1}[a_{t+1} > m_t].$$

Everything computed at step t uses values up to x_t only.

2 The null: what the series' own record implies

In plain words

Before anything else the engine asks: how often has this event happened on this series? "How often lately" and "how often ever" can differ, so it keeps seven running tallies, each forgetting the past at a different speed, and trusts each in proportion to how well it has predicted. The trusted blend is the **null**, q_0: the series' usual odds.

The null never makes a call. It is what every call is measured against, so that HEX gets no credit for what was already obvious.

For half-lives $h \in \{32, 64, 128, 256, 512, 1024, \infty\}$ with $\rho_h = 2^{-1/h}$:

$$U_h \leftarrow \rho_h U_h + y, \qquad N_h \leftarrow \rho_h N_h + 1, \qquad e_h = \frac{U_h + 1}{N_h + 2} \text{ (kept within } [0.01, 0.99]).$$

For MOVE one more estimate is added, the position rule: if A of the n values so far lie above the present one, $e_{\text{pos}} = (A+1)/(n+1)$, the exact probability that the next value is higher when the order of the values carries no information. Each estimate has a log-trust ℓ_k, starting at zero:

$$\ell_k \leftarrow \ell_k + \ln e_k(y), \qquad v_k = \frac{e^{\ell_k}}{\sum_j e^{\ell_j}}, \qquad q_0 = \sum_k v_k e_k,$$

with $e_k(y) = e_k$ if $y = 1$ and $1 - e_k$ otherwise. This is a Bayesian mixture with a uniform prior: on any series its total log loss exceeds that of the best single memory by at most $\ln M$, $M = 7$ or 8 (Vovk, 1990).

In code. base44/shared/observer.ts, from line 60: the seven memories, the position rule, and their mixture

```ts
export function observe(returns: number[], read: ReadName, detail = false): { baseline: Float64Array; memories: View[]; history?: View[][] } {
  const n = returns.length;
  const memories: View[] = [];
  const history: View[][] | undefined = detail ? [] : undefined;
  const out = new Float64Array(n).fill(NaN);
  const a = returns.map(Math.abs);
  // MOVE: rank of the present level among all levels so far
  const rankMax = new Float64Array(n);
  // RANGE: median of |change| through step i
  const med = new Float64Array(n).fill(NaN);
  // The level is rounded to nine decimals before it is ranked, so that a level returning exactly to an
  // earlier level counts as a tie whatever arithmetic produced the changes.
  if (read === 'MOVE') { const s = new Sorted(); let x = 0; for (let i = 0; i < n; i++) { x += returns[i]; rankMax[i] = s.push(Math.floor(x * 1e9 + 0.5) / 1e9); } }
  else { const s = new Sorted(); for (let i = 0; i < n; i++) { s.push(a[i]); if (s.size >= MIN_MEDIAN) med[i] = s.median(); } }

  const H = HALF_LIVES.length;
  const rho = HALF_LIVES.map((h) => (h === null ? 1 : 0.5 ** (1 / h)));
  const U = new Float64Array(H), N = new Float64Array(H);
  const ne = H + (read === 'MOVE' ? 1 : 0);
  const ll = new Float64Array(ne), ex = new Float64Array(ne);
  for (let i = FIRST; i < n; i++) {
    if (read === 'RANGE' && Number.isNaN(med[i])) continue;
    for (let h = 0; h < H; h++) ex[h] = clip((U[h] + 1) / (N[h] + 2));
    if (read === 'MOVE') ex[H] = clip((i + 1 - rankMax[i] + 1) / (i + 2));
    let m = -Infinity;
    for (let k = 0; k < ne; k++) if (ll[k] > m) m = ll[k];
    let num = 0, den = 0;
    for (let k = 0; k < ne; k++) { const w = Math.exp(ll[k] - m); num += w * ex[k]; den += w; }
    out[i] = num / den;
    if (i === n - 1 || history) {
      const now: View[] = [];
      for (let k = 0; k < ne; k++) now.push({ label: k < H ? MEMORY_LABELS[k] : 'position', p: ex[k], w: Math.exp(ll[k] - m) / den });
      if (history) history[i] = now;
      if (i === n - 1) { memories.push(...now); break; }
    }
    const y = read === 'MOVE' ? (returns[i + 1] > 0 ? 1 : 0) : (a[i + 1] > med[i] ? 1 : 0);
    for (let k = 0; k < ne; k++) ll[k] += Math.log(y ? ex[k] : 1 - ex[k]);
    for (let h = 0; h < H; h++) { U[h] = rho[h] * U[h] + y; N[h] = rho[h] * N[h] + 1; }
  }
  return { baseline: out, memories, ...(history ? { history } : {}) };
}
```

3 Where the present sits: the coordinates Z

In plain words

Is today's value high or low *compared with its recent surroundings*? Take the last 8 values and ask where today's ranks among them: lowest, highest, or in between. Do the same for the last 2, 4, 16, 32 and 64 values, and for the whole history. That gives seven numbers between -1 (the lowest in its window) and $+1$ (the highest). They are computed for two things: the level of the series, and the size of its latest change.

A bitcoin price, a temperature and a finishing position all become the same kind of number: *where does this sit relative to what came before?*

Example: today's value is the 7th smallest of the last 8. Its coordinate at that scale is $2(7 - \frac{1}{2})/8 - 1 = 0.625$.

For window $w \in \{2, 4, 8, 16, 32, 64\}$ let $R_{t,w}$ be the midrank of v_t among $v_{t-w+1}, \ldots, v_t$ (ties share the average of their positions), and $R_{t,\infty}$ its midrank among $v_1, \ldots, v_t$:

$$u_{t,w} = \frac{R_{t,w} - \frac{1}{2}}{w}, \qquad z_{t,w} = 2u_{t,w} - 1 \in (-1, 1).$$

A window that is not yet full gives 0. Values are rounded to nine decimals before ranking, so that a level returning exactly to an earlier level ranks as a tie whatever arithmetic produced it. Two observables are ranked: the level $v_t = \sum_{s \le t} r_s$ and the size $v_t = a_t$. For MOVE the level is the read's own observable and the size is the other; for RANGE the reverse. $Z_t \in \mathbb{R}^{14}$ is the own seven followed by the other seven.

If the series were exchangeable inside a window, $u_{t,w}$ would be uniform: it is a probability integral transform, which is why the same coordinates serve every kind of series. The sign $\mathbb{1}[z > 0]$ is a binary line; earlier engines kept only the sign.

In code. base44/shared/hex7.ts, from line 59: the coordinates

```
export function ranks(values: number[]): Float64Array[] {
  const v = values.map((x) => Math.floor(x * 1e9 + 0.5) / 1e9);
  const n = v.length, out: Float64Array[] = [];
  const sorted: number[] = [];
  for (let i = 0; i < n; i++) {
    const z = new Float64Array(NZ), x = v[i];
    for (let k = 0; k < WINDOWS.length; k++) {
      const w = WINDOWS[k];
      if (i < w - 1) continue;
      let less = 0, equal = 0;
      for (let j = i - w + 1; j <= i; j++) { if (v[j] < x) less++; else if (v[j] === x) equal++; }
      z[k] = 2 * (less + (equal + 1) / 2 - 0.5) / w - 1;
    }
    // all history: insert x, count values below it and equal to it
    let lo = 0, hi = sorted.length;
    while (lo < hi) { const m = (lo + hi) >> 1; if (sorted[m] < x) lo = m + 1; else hi = m; }
    let up = lo; while (up < sorted.length && sorted[up] === x) up++;
    sorted.splice(lo, 0, x);
    z[NZ - 1] = 2 * (lo + (up - lo + 2) / 2 - 0.5) / (i + 1) - 1;
    out.push(z);
  }
  return out;
}
```

4 What HEX learns: the residual

In plain words

The null says 55%. HEX asks whether where the value sits changes that. It learns one number per coordinate: how much being high at that scale has pushed the odds up or down on this series. All of it starts at zero, so on a new series HEX starts exactly at the null and departs from it only as evidence arrives. The adjustment is made in odds, not percentage points, so the result can never leave 0–100%.

Example: the null says 55%, which is odds of $0.55/0.45 = 1.22$. HEX has learned a coefficient of -0.3 at the 8-step scale, and today's coordinate there is 0.625. The odds are multiplied by $e^{-0.3 \times 0.625} = 0.83$, giving 1.01, or 50.3%: on this series, sitting high in the recent range has been followed by fewer rises.

Two models: own ranks only ($\beta \in \mathbb{R}^7$) and own with the other observable ($\beta \in \mathbb{R}^{14}$). Each is Bayesian logistic regression with offset logit q_0, updated one outcome at a time by the second-order (assumed-density) step. With prior $\beta \sim N(0, \tau I)$, covariance P starting at τI, and $q = \sigma(\operatorname{logit} q_0 + \beta^\top Z)$ kept within $[0.01, 0.99]$:

$$P \leftarrow P - \frac{q(1-q)\, PZZ^\top P}{1 + q(1-q)\, Z^\top PZ}, \qquad \beta \leftarrow \beta + PZ\,(y - q).$$

There is no learning rate to choose. The one free quantity, the prior scale τ, is a family: nine copies run at $\tau \in \{2^{-4}, \ldots, 2^4\}$ and are mixed by likelihood exactly as in Section 2. The model's probability is that mixture; the coefficients in force are the copies' coefficients averaged by their weights.

In code. base44/shared/hex7.ts, from line 109: one residual model: predict, update, and the coefficients in force

```
class Residual {
```

```
  P: Float64Array[]; th: Float64Array[]; ll: Float64Array = new Float64Array(T); q: Float64Array = new Float64Array(T); x: Float64Array = new Float64Array(D);
  constructor() {
    this.P = PRIOR_SCALES.map((t) => { const m = new Float64Array(D * D); for (let i = 0; i < D; i++) m[i * D + i] = t; return m; });
    this.th = PRIOR_SCALES.map(() => new Float64Array(D));
  }
  mix(): Float64Array { let m = -Infinity; for (const v of this.ll) if (v > m) m = v; const w = new Float64Array(T); let z = 0; for (let t = 0; t < T; t++) { w[t] = Math.exp(this.ll[t] - m); z += w[t]; } for (let t = 0; t < T; t++) w[t] /= z; return w; }
  predict(x: Float64Array, offset: number): number {
    this.x = x; const w = this.mix(); let p = 0;
    for (let t = 0; t < T; t++) { let s = offset; const th = this.th[t]; for (let d = 0; d < D; d++) s += th[d] * x[d]; this.q[t] = clip(1 / (1 + Math.exp(-s))); p += w[t] * this.q[t]; }
    return p;
  }
  update(y: number) {
    const x = this.x, Px = new Float64Array(D);
    for (let t = 0; t < T; t++) {
      const q = this.q[t], P = this.P[t], th = this.th[t];
      this.ll[t] += Math.log(y ? q : 1 - q);
      const wt = q * (1 - q); let xPx = 0;
      for (let i = 0; i < D; i++) { let s = 0; for (let j = 0; j < D; j++) s += P[i * D + j] * x[j]; Px[i] = s; }
      for (let i = 0; i < D; i++) xPx += Px[i] * x[i];
      const k = wt / (1 + wt * xPx);
      for (let i = 0; i < D; i++) for (let j = 0; j < D; j++) P[i * D + j] -= k * Px[i] * Px[j];
      for (let i = 0; i < D; i++) { let s = 0; for (let j = 0; j < D; j++) s += P[i * D + j] * x[j]; th[i] += s * (y - q); }
    }
  }
  /** The coefficients in force: the prior scales' coefficients averaged by their weights. */
  beta(): number[] { const w = this.mix(); return Array.from({ length: D }, (_, d) => this.th.reduce((s, th, t) => s + w[t] * th[d], 0)); }
}
```

5 HEX's call: reversible influence

In plain words

Three voices answer every question: the null alone, the model that reads the read's own coordinates, and the model that also reads the other observable. HEX's call is their weighted average. A voice gains influence by giving high probability to what actually happened.

Influence is never locked in. After every outcome a small slice of everyone's influence is shared out equally, so a voice that has been wrong for a long time can come back quickly when the series changes. What a voice has learned is never erased; only its present say in the call moves.

For experts $V = (q_0, m_1, m_2)$ and a share rate α (fixed share; Herbster and Warmuth, 1998):

$$w_k \leftarrow (1-\alpha)\,\frac{w_k V_k(y)}{\sum_j w_j V_j(y)} + \frac{\alpha}{3}, \qquad p = \sum_\alpha o_\alpha \sum_k w_{\alpha,k} V_k.$$

Five rates $\alpha \in \{0, 0.001, 0.005, 0.02, 0.05\}$ run side by side and are mixed by likelihood with weights o_α; $\alpha = 0$ is the plain Bayesian mixture, so the plain mixture is one of the things HEX can fall back on, at a cost of at most $\ln 5$ in total log loss.

In code. base44/shared/hex7.ts, from line 84: the mixture with reversible influence

```
class Share {
  w: Float64Array[]; ll: Float64Array; q: Float64Array; o: Float64Array;
  constructor(private n: number) {
    this.w = SHARE_RATES.map(() => new Float64Array(n).fill(1 / n));
    this.ll = new Float64Array(SHARE_RATES.length); this.q = new Float64Array(SHARE_RATES.length); this.o = new Float64Array(SHARE_RATES.length);
  }
  private outer() { let m = -Infinity; for (const v of this.ll) if (v > m) m = v; let z = 0; for (let a = 0; a < this.o.length; a++) { this.o[a] = Math.exp(this.ll[a] - m); z += this.o[a]; } for (let a = 0; a < this.o.length; a++) this.o[a] /= z; }
  predict(P: number[]): number {
    this.outer(); let p = 0;
    for (let a = 0; a < this.w.length; a++) { let q = 0; for (let e = 0; e < this.n; e++) q += this.w[a][e] * P[e]; this.q[a] = q; p += this.o[a] * q; }
    return p;
  }
  update(P: number[], y: number) {
    for (let a = 0; a < this.w.length; a++) {
```

```
      this.ll[a] += Math.log(y ? this.q[a] : 1 - this.q[a]);
      const w = this.w[a]; let z = 0;
      for (let e = 0; e < this.n; e++) { w[e] *= y ? P[e] : 1 - P[e]; z += w[e]; }
      for (let e = 0; e < this.n; e++) w[e] = (1 - SHARE_RATES[a]) * (w[e] / z) + SHARE_RATES[a] / this.n;
    }
  }
  /** Each expert's present share of the call. */
  influence(): number[] { this.outer(); return Array.from({ length: this.n }, (_, e) => this.w.reduce((s, w, a) => s + this.o[a] * w[e], 0)); }
}
```

6 The walk: one pass, in order

In plain words

All of the above happens in one loop over the series, and the order inside the loop is the whole discipline: *predict; then the outcome arrives; then learn*. Nothing is ever learned from an outcome before the call for it has been made.

In code. base44/shared/hex7.ts, from line 156: predict, outcome, learn, for every step of a series

```
export function walkHex7(returns: number[], read: ReadName, baseline: Float64Array, detail = false): { steps: Hex7Step[]; final: Hex7Final | null } {
  const n = returns.length, a = returns.map(Math.abs);
  const level: number[] = []; let acc = 0; for (let i = 0; i < n; i++) { acc += returns[i]; level.push(acc); }
  const med = new Float64Array(n).fill(NaN); const sorted: number[] = [];
  for (let j = 0; j < n; j++) {
    let lo = 0, hi = sorted.length; while (lo < hi) { const m = (lo + hi) >> 1; if (sorted[m] < a[j]) lo = m + 1; else hi = m; }
    sorted.splice(lo, 0, a[j]);
    if (sorted.length >= MIN_MEDIAN) { const m = sorted.length; med[j] = m % 2 ? sorted[(m - 1) / 2] : (sorted[m / 2 - 1] + sorted[m / 2]) / 2; }
  }
  const Zown = ranks(read === 'MOVE' ? level : a), Zother = ranks(read === 'MOVE' ? a : level);
  const R = [new Residual(), new Residual()], A = new Share(3);
  const x0 = new Float64Array(D), x1 = new Float64Array(D);
  const steps: Hex7Step[] = []; let final: Hex7Final | null = null;
  for (let i = FIRST; i < n; i++) {
    if (read === 'RANGE' && Number.isNaN(med[i])) continue;
    const q0 = baseline[i];
    if (!Number.isFinite(q0)) continue;
    // predict
    for (let d = 0; d < NZ; d++) { x0[d] = Zown[i][d]; x1[d] = Zown[i][d]; x1[NZ + d] = Zother[i][d]; }
    const off = logit(q0), m0 = R[0].predict(x0, off), m1 = R[1].predict(x1, off), V = [q0, m0, m1], p = A.predict(V);
    if (i === n - 1) { final = { p, b: q0, rank: m0, cross: m1, beta: R[1].beta(), influence: A.influence() }; break; }
    // the outcome arrives; learn only now
    const y = read === 'MOVE' ? (returns[i + 1] > 0 ? 1 : 0) : (a[i + 1] > med[i] ? 1 : 0);
    steps.push(detail ? { i, p, b: q0, y, rank: m0, cross: m1, beta: R[1].beta(), influence: A.influence() } : { i, p, b: q0, y });
    R[0].update(y); R[1].update(y); A.update(V, y);
  }
  return { steps, final };
}
```

7 Strength: restated from the record, never changing the call

In plain words

A call has a side (yes or no) and a strength (how far from 50%). The side is HEX's and is never changed. The strength is checked against history: of all earlier calls stated at about 70%, how many were right? If 66% were, a new 70% is shown as 66%. The shown strength can never go below 50%, so the side cannot flip.

With $a = \mathbb{1}[p \geq \frac{1}{2}]$ and $s = \max(p, 1-p)$, let G be the isotonic regression of $\mathbb{1}[\text{call was right}]$ on s over calls whose outcomes are known, pooled across tracked series and refitted nightly (at least 2,000 calls). Then $s' = \min(0.99, \max(0.5001, G(s)))$ and the shown probability is s' if $a = 1$, else $1 - s'$. A map allowed to change the side was tested and rejected: the calls it changed were right less often afterwards.

In code. base44/shared/isotonic.ts, from line 66: the strength map

```
export function fitStrength(ps: number[], ys: number[]): (s: number) => number {
  return fitIsotonic(ps.map(strength), ps.map((p, i) => ((isUp(p) ? 1 : 0) === ys[i] ? 1 : 0)));
}
/** The same map as knots, to store. */
export function strengthKnots(ps: number[], ys: number[]): Knots {
  return isotonicKnots(ps.map(strength), ps.map((p, i) => ((isUp(p) ? 1 : 0) === ys[i] ? 1 : 0)));
}
/** Restate the strength of a call; the side stays where HEX put it. */
export function restate(p: number, map: (s: number) => number): number {
  const s = Math.min(STRENGTH_CEIL, Math.max(STRENGTH_FLOOR, map(strength(p))));
  return isUp(p) ? s : 1 - s;
}
```

8 The measure: information gain

In plain words

Counting right and wrong treats a 51% call and a 99% call the same. The engine is judged instead by how much probability it gave to what actually happened, compared with how much the null gave. If HEX said 62% and the null said 55% and the event happened, HEX gained $\log_2(0.62/0.55) = 0.17$ bits; if it did not happen, HEX lost 0.24 bits.

Added up over every call, this is the information HEX has contributed beyond the obvious. Positive and clear of zero means a **verified edge**. Not clear of zero means **no verified edge**, and the product says so.

$$g = \ln p(y) - \ln q_0(y), \qquad G_T = \sum_{t \le T} g_t, \qquad \bar{g} = \frac{G_T}{T \ln 2} \text{ bits per call.}$$

This is a proper score: its expectation is greatest when the true probability is stated. Calls on the same day move together, so days are the unit of uncertainty. For day totals S_d over n_d calls, D days, $N = \sum_d n_d$:

$$m = \frac{\sum_d S_d}{N}, \qquad \text{se} = \frac{1}{N}\sqrt{\frac{D}{D-1}\sum_d (S_d - m\,n_d)^2}, \qquad \text{interval} = m \pm 1.96 \text{ se}\,.$$

Only four running sums are needed ($\sum S_d$, $\sum S_d^2$, $\sum S_d n_d$, $\sum n_d^2$), so every decision uses the whole record without rereading it.

In code. base44/shared/cycle.ts, from line 70: the log score, information gain, and the day-clustered interval

```
export const logScore = (p: number, y: number) => Math.log(y ? Math.min(0.99, Math.max(0.01, p)) : 1 - Math.min(0.99,
    ↪ Math.max(0.01, p)));
/** Information gain of p over q on one outcome, in nats: ln p(y) - ln q(y). Divide by ln 2 for bits. */
export const informationGain = (p: number, q: number, y: number) => logScore(p, y) - logScore(q, y);

/**
 * Running sums for a per-call mean with days as the unit of uncertainty:
 * calls made on the same day move together, so a day's calls enter as one
 * cluster. For day totals S_d over n_d calls, the mean is m = sum S_d / sum n_d
 * and its standard error is sqrt( D/(D-1) * sum (S_d - m n_d)^2 ) / sum n_d.
 */
export interface Clustered { n: number; days: number; s: number; ss: number; sn: number; nn: number }
export const EMPTY_CLUSTERED: Clustered = { n: 0, days: 0, s: 0, ss: 0, sn: 0, nn: 0 };
export function addDay(c: Clustered, dayTotal: number, dayCalls: number): Clustered {
  return { n: c.n + dayCalls, days: c.days + 1, s: c.s + dayTotal, ss: c.ss + dayTotal * dayTotal, sn: c.sn + dayTotal *
    ↪ dayCalls, nn: c.nn + dayCalls * dayCalls };
}
/** The mean per call and its interval at z standard errors; null until there are two days. */
export function clusteredMean(c: Clustered, z = 1.96): { mean: number; low: number; high: number } | null {
  if (c.days < 2 || !c.n) return null;
  const m = c.s / c.n, se = Math.sqrt(Math.max(0, c.ss - 2 * m * c.sn + m * m * c.nn) * c.days / (c.days - 1)) / c.n;
  return { mean: m, low: m - z * se, high: m + z * se };
}
```

Recorded beside it for every call: **closeness** $c = 1 - (p - y)^2$ and the identity

$$c = 0.75 + \underbrace{[c(q_0) - 0.75]}_{\text{null}} + \underbrace{[c(p) - c(q_0)]}_{\text{what HEX added}} + \underbrace{[c(p_{\text{shown}}) - c(p)]}_{\text{strength}};$$

reward $\max(0, c - \bar{c})$, the progress over the version's own average closeness on that series, never negative; **lesson** $\max(0, \bar{c} - c)$, kept for analysis and subtracted from nothing (no decision is taken on reward, because cutting off the downside would pay for boldness); and the **reason for a miss**.

In code. base44/shared/cycle.ts, from line 158: everything recorded for one scored call

```
export function scoreCall(stated: number, views: Views, outcome: number, record: Record_): Scored {
  const c = closeness(stated, outcome);
  const before = record.n ? record.mean : START_CLOSENESS;
  return {
    outcome,
    right: sideOf(stated) === outcome,
    log_score: logScore(stated, outcome),
    info_gain: informationGain(stated, views.baseline, outcome),
    closeness: c,
    closeness_baseline: closeness(views.baseline, outcome),
    reward: rewardOf(c, before),
    lesson: lessonOf(c, before),
    reason: reasonFor(stated, views, outcome),
    account: accountOf(stated, views, outcome),
    record: { n: record.n + 1, mean: record.n ? record.mean + (c - record.mean) / (record.n + 1) : c },
  };
}
```

9 The reason for a miss

In plain words

At the moment of the call the engine writes down not only its answer but what every voice said on its own: each of the seven tallies, the position rule, and each of its two models. When a call turns out wrong, those notes show where it went wrong, with no hindsight, because they were written before the outcome.

structure	the null was on the right side; what HEX added turned the call.
change	some memories were right and they were, on average, the shorter ones: the series had changed.
blip	some memories were right and they were, on average, the longer ones: the recent stretch was unusual.
outvoted	no memory was right, but the position rule or one of HEX's models was, with too little influence.
unseen	nothing the engine remembers about this series pointed the right way.

In code. base44/shared/cycle.ts, from line 109: the reason, from what was written down before the outcome

```
export function reasonFor(stated: number, views: Views, outcome: number): Reason | null {
  const right = (p: number) => sideOf(p) === outcome;
  if (right(stated)) return null;
  if (right(views.baseline)) return 'structure';
  const lengths = (views.memories || []).filter((m) => m.label.startsWith('memory:'));
  const at = (ok: boolean) => lengths.map((m, i) => (right(m.p) === ok ? i : -1)).filter((i) => i >= 0);
  const had = at(true), hadNot = at(false);
  if (had.length && hadNot.length) {
    const mean = (v: number[]) => v.reduce((s, x) => s + x, 0) / v.length;
    return mean(had) < mean(hadNot) ? 'change' : 'blip'; // memories are listed shortest first
  }
  const others = [...(views.memories || []).filter((m) => !m.label.startsWith('memory:')), ...(views.readings || [])];
  if (had.length || others.some((v) => right(v.p))) return 'outvoted';
  return 'unseen';
}
```

10 The learning loop

In plain words

Every night, for every series: see what happened; score last night's calls; let memory take it in; make tonight's calls; write them down before their outcomes exist. Then, across all series: refit the strength

map, decide which version people see, and decide which reads have a verified edge.

The engine keeps nothing between nights. Each night it walks every series again from its first value, so what it knows is exactly what one pass through that series' history teaches it. There is no stored state to corrupt.

1. **Reveal.** For each open call whose next bar has printed, compute y.
2. **Score.** Information gain, closeness, reward, lesson, reason (Sections 8 and 9).
3. **Learn.** The walk of Section 6 takes every outcome into U, N, ℓ, into P, β and the prior-scale weights, and into the influences w, o.
4. **Predict.** Tonight's calls from both versions, with what each voice said and, for 7.0, the coefficients and influences in force.
5. **Commit.** Written to the ledger before the outcome exists.
6. **Strength map.** Refit G per read and version from every scored step.
7. **Which version is shown.** 7.0 and the frozen 6.5 run side by side. On the same calls, with m and its interval for $\ln p_{7.0}(y) - \ln p_{6.5}(y)$: 7.0 is switched in once there are 300 paired calls over 60 days, the interval is above zero, and 7.0 is right at least as often. It is taken down when the interval is below zero, or it is right less often by a margin clear of zero.
8. **Verified edge.** For the version shown, m and its interval for information gain over the null. The status starts from the replay (market move size verified, market direction not). The ledger's own record changes it only when it is decisive: to verified when the lower bound is above zero, to not verified when the upper bound is below zero.

In code. base44/functions/calibrateCognition/entry.ts, from line 52: steps 1 to 5 for every tracked series, in that order

```
52      for (const ticker of tickers) {
53        const t0 = Date.now();
54        let md = null;
55        try { md = await fetchYahooReturns(ticker, HISTORY_DAYS); } catch { failed.push(ticker); continue; }
56        const bars = { closes: md.completedCloses, timestamps: md.timestamps.slice(0, md.completedBars) };
57        scored += await scoreOpenCalls(base44, ticker, bars);
58        const run = runEngines(md.returns, live);
59        if (!run) { failed.push(ticker); continue; }
60        logged += await logCalls(base44, rowsFor(ticker, bars, md.returns, run, 'nightly'));
61        // Each version's own scored steps on this series. Every outcome in them
62        // is already known, so pooling them leaks nothing forward.
63        for (const e of ENGINES) for (const read of READS) pooled[e][read].push(...run.steps[e][read]);
64        timings[ticker] = Date.now() - t0;
65      }
```

In code. base44/shared/ledger.ts, from line 284: steps 7 and 8: the version rule and the verified-edge test

```
284 export function decide(read: ReadName, t: Tally, current: EngineId): ReadStatus {
285   const st: ReadStatus = {
286     read, live_engine: current, paired_calls: t.calls, paired_days: t.days,
287     gain: null, gain_low: null, gain_high: null,
288     bits_control: null, bits_control_low: null, bits_control_high: null, bits_candidate: null, bits_candidate_low: null,
        ↪ bits_candidate_high: null, verified: null,
289     hit_control: null, hit_candidate: null, hit_baseline: null, stated_control: null, stated_candidate: null,
        ↪ closeness_control: null, closeness_candidate: null,
290     progress_control: null, progress_candidate: null, lesson_control: null, lesson_candidate: null,
291     reasons_control: { ...(t.reasons_control || {}) }, reasons_candidate: { ...(t.reasons_candidate || {}) }, decision:
        ↪ 'hold', reason: '',
292   };
293   if (!t.calls) { st.reason = 'no scored calls yet'; return st; }
294   const r4 = (v: number) => Number(v.toFixed(4)), r5 = (v: number) => Number(v.toFixed(5)), per = (v: number) => v / t.
        ↪ calls;
295   const v = clusteredMean(t.versus, Z), c0 = clusteredMean(t.over_null_control, Z), k0 = clusteredMean(t.
        ↪ over_null_candidate, Z), h = clusteredMean(t.hits, Z);
296   if (v) { st.gain = r5(v.mean); st.gain_low = r5(v.low); st.gain_high = r5(v.high); }
297   if (c0) { st.bits_control = r5(c0.mean / LN2); st.bits_control_low = r5(c0.low / LN2); st.bits_control_high = r5(c0.
        ↪ high / LN2); }
298   if (k0) { st.bits_candidate = r5(k0.mean / LN2); st.bits_candidate_low = r5(k0.low / LN2); st.bits_candidate_high = r5
        ↪ (k0.high / LN2); }
299   st.hit_control = r4(per(t.hits_control)); st.hit_candidate = r4(per(t.hits_candidate)); st.hit_baseline = r4(per(t.
        ↪ hits_baseline));
300   st.stated_control = r4(per(t.stated_control)); st.stated_candidate = r4(per(t.stated_candidate));
301   st.closeness_control = r5(per(t.closeness_control)); st.closeness_candidate = r5(per(t.closeness_candidate));
```

```
  st.progress_control = r5(per(t.progress_control)); st.progress_candidate = r5(per(t.progress_candidate));
  st.lesson_control = r5(per(t.lesson_control)); st.lesson_candidate = r5(per(t.lesson_candidate));

  const enough = t.calls >= MIN_PAIRED_CALLS && t.days >= MIN_PAIRED_DAYS && !!v;
  const bits = (x: number) => (x / LN2 * 1000).toFixed(1);
  const span = v ? `[${bits(v.low)}, ${bits(v.high)}]` : '';
  if (!enough) {
    st.reason = `${t.calls} of ${MIN_PAIRED_CALLS} paired calls over ${t.days} of ${MIN_PAIRED_DAYS} days: no decision
      ↪ yet`;
  } else if (current === CONTROL_ENGINE && v!.low > 0 && t.hits_candidate < t.hits_control) {
    st.reason = `7.0 gains ${bits(v!.mean)} thousandths of a bit per call over 6.5 but is right less often (${(st.
      ↪ hit_candidate! * 100).toFixed(1)}% against ${(st.hit_control! * 100).toFixed(1)}%), so 6.5 stays`;
  } else if (current === CONTROL_ENGINE && v!.low > 0) {
    st.decision = 'promote'; st.live_engine = CANDIDATE_ENGINE;
    st.reason = `7.0 gains ${bits(v!.mean)} thousandths of a bit per call over 6.5 ${span}; the interval excludes zero`;
  } else if (current === CANDIDATE_ENGINE && v!.high < 0) {
    st.decision = 'demote'; st.live_engine = CONTROL_ENGINE;
    st.reason = `6.5 gains ${bits(-v!.mean)} thousandths of a bit per call over 7.0; the interval excludes zero`;
  } else if (current === CANDIDATE_ENGINE && h && h.high < 0) {
    st.decision = 'demote'; st.live_engine = CONTROL_ENGINE;
    st.reason = `7.0 was right less often than 6.5 (${(st.hit_candidate! * 100).toFixed(1)}% against ${(st.hit_control!
      ↪ * 100).toFixed(1)}%); the interval excludes zero`;
  } else if (current === CONTROL_ENGINE) {
    st.reason = `7.0 over 6.5: ${bits(v!.mean)} thousandths of a bit per call ${span}: not clearly ahead, so 6.5 stays`;
  } else {
    st.reason = `7.0 over 6.5: ${bits(v!.mean)} thousandths of a bit per call ${span}: not clearly behind, so 7.0 stays
      ↪ `;
  }
  // a verified edge: the version that will be shown gains over the null by a margin clear of zero.
  // The ledger's own record settles it only when it is decisive either way; until then what was
  // established before it stands (the caller keeps the earlier status when this is null).
  const shown = st.live_engine === CANDIDATE_ENGINE ? k0 : c0;
  st.verified = enough && shown ? (shown.low > 0 ? true : shown.high < 0 ? false : null) : null;
  return st;
}
```

11 The precision metrics the loop keeps

In plain words

The loop does not just learn the series; it keeps a running account of how well it is doing, per read, and every decision above is read off that account. These are the numbers to monitor.

One `EngineStatus` row per read:

`bits_candidate`, `_low`, `_high`	7.0's information gain over the null, bits per call, with its 95% interval
`bits_control`, `_low`, `_high`	the same for 6.5
`gain`, `gain_low`, `gain_high`	7.0 over 6.5 on the same calls, nats per call
`verified`	whether the shown version has a verified edge over the null
`hit_candidate`, `hit_control`, `hit_baseline`	share of calls right: 7.0, 6.5, and what the null alone gets (expected)
`stated_candidate`, `stated_control`	mean stated strength, to set beside the share right
`closeness_*`, `progress_*`, `lesson_*`	mean closeness, reward and lesson per call
`reasons_candidate`, `reasons_control`	misses counted by reason
`paired_calls`, `paired_days`, `live_engine`, `decision`, `reason`	the record's size and the version decision in words
`tally`	the running sums behind all of the above

Per call, in `CallLedger`: `info_gain`, `log_score`, `closeness`, `reward`, `lesson`, `reason`, the three parts of closeness, `views` (what each voice said) and `learned` (7.0's coefficients and influences).

What those metrics read after ten years

The product's own code and ledger functions were run night by night on 17 market series from January 2016 to September 2026, starting with an empty ledger and an engine that knew nothing (46,461 series-

nights; `validation/timesim`). The final `EngineStatus` rows:

	Move size (RANGE)	Direction (MOVE)
Version shown	7.0	6.5
Verified edge	yes	no
Paired calls / days	46,436 / 3,892	46,436 / 3,892
7.0: bits per call over the null	+0.00421	+0.00043
95% interval	[+0.00306, +0.00535]	[−0.00038, +0.00123]
6.5: bits per call over the null	+0.00036	+0.00044
7.0 over 6.5, nats per call	+0.00266	−0.00001
95% interval	[+0.00191, +0.00342]	[−0.00034, +0.00032]
7.0 right / 6.5 right	57.9% / 57.4%	51.4% / 51.3%

Version and status changes over the ten years: 2016-04-04 direction: 6.5 switched in; 2016-05-22 move size: 6.5 switched in; 2017-07-03 move size: not verified; 2019-05-20 move size: 7.0 switched in; 2020-03-25 move size: verified. In this cold start both reads begin on 7.0, which is behind for its first weeks on a series it has never seen; the rule takes it down within months, switches move size back to 7.0 once the record is decisive, and verifies it in 2020. In the product the engine is not cold: each series is walked from the start of its history.

Why calls missed, over the same run:

	Misses	unseen	blip	change	structure	outvoted
Move size, 7.0	19,570	47.0%	32.0%	6.2%	9.2%	5.7%
Move size, 6.5	19,775	51.9%	34.5%	6.9%	4.3%	2.5%
Direction, 7.0	22,581	17.9%	7.2%	21.5%	14.9%	38.5%
Direction, 6.5	22,629	16.5%	8.2%	21.5%	10.6%	43.1%

Where closeness came from, by the identity of Section 8 (mean per call):

	Closeness	from the null	from what HEX added	from strength
Move size, 7.0	0.7595	+0.0080	+0.0015	0.0000
Move size, 6.5	0.7582	+0.0080	+0.0002	0.0000
Direction, 7.0	0.7501	−0.0001	0.0000	+0.0001
Direction, 6.5	0.7501	−0.0001	0.0000	+0.0001

After every miss whose reason was *change* or *blip*, trust in the memories that had been on the right side rose by the next call: 13,934 such misses, 100% of them.

The full results of the one-pass replay on 43 series, including series never used before, are in the companion document, *HEX [PI] 7.0: the ten-year replay and how to run the system*.

12 What is not claimed

- No guarantee of accuracy on any future series. Finite history cannot prove the future.
- The interval rules use a normal approximation over days. They are decision rules, not proofs.
- Next-day market direction has no verified edge. Nothing in 7.0 fixes that, and the product does not pretend otherwise.
- Interaction terms, calibration that may change a call, and an interference (amplitude) mixture were each tested and added no information; none is in 7.0. Text, letter-number and ontology layers are not part of 7.0.

References

Ayer, Brunk, Ewing, Reid and Silverman (1955). An empirical distribution function for sampling with incomplete information. *Ann. Math. Statist.* 26.

Dawid (1984). Statistical theory: the prequential approach. *J. Roy. Statist. Soc. A* 147.
Gneiting and Raftery (2007). Strictly proper scoring rules, prediction, and estimation. *J. Amer. Statist. Assoc.* 102.
Herbster and Warmuth (1998). Tracking the best expert. *Machine Learning* 32.
Opper (1998). A Bayesian approach to on-line learning. In *On-line Learning in Neural Networks*, Cambridge University Press.
Vovk (1990). Aggregating strategies. *Proc. COLT*.

© 2026 Evrica Technologies Inc. · anna@evrica.tech · source: github.com/EvricaTechnologiesInc/HEX-PI